

George Berar

85 Followers

Lists

About

Follow



SpringBoot: Standardized API Exception Handling



George Berar Dec 16, 2021 · 8 min read

Handling API exceptions uniformly can be tricky and challenging sometimes so in today's article I will talk about how to create a standardized API exception handling mechanism.

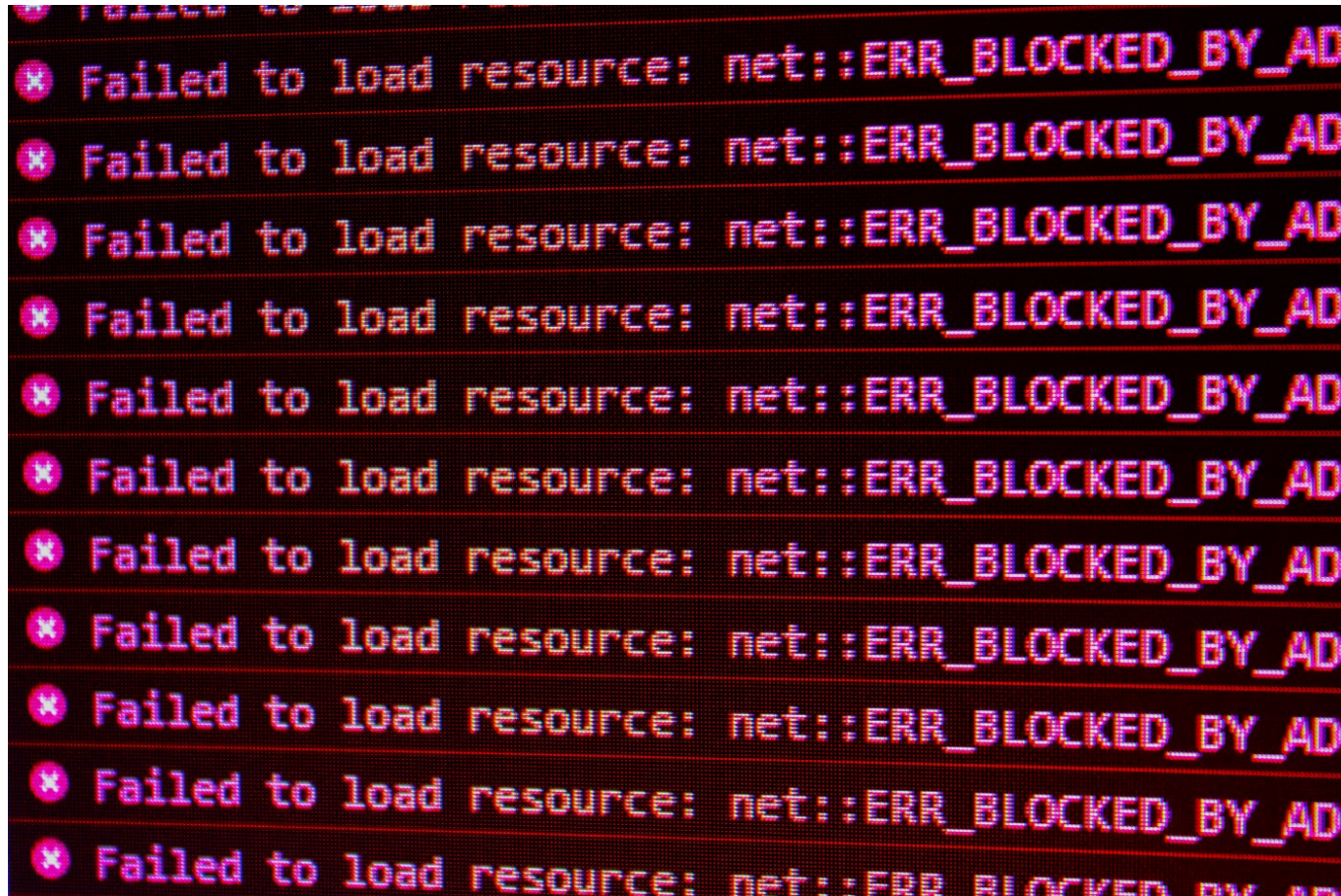


Photo by [David Pupaza](#) on [Unsplash](#)

Before we start you can find the entire [code here](#). Ok, let's dive in!

Background

In my past projects I've seen many different approaches on handling exceptions at API level and returning meaningful responses; from using 200 OK status code in case of an error (yeah, you read that right) to returning no response body at all. Some of them were a little bit 'more advanced' and leveraged the built-in Spring mechanism with some level of customization to give more information about what happened but they were poorly designed.

Giving a meaningful feedback to the caller of your API in case of an error is as important as the success response. It must contain a minimal set of details which should express what happened and it needs to be properly formatted as JSON.

The Problem

A poorly designed API exception handling mechanism makes error response understanding and processing much more difficult than it should actually be.

If the error response does not show what happened, even at a basic level, it would be twice as hard for a developer to find out what's the problem and come up with a solution. The continuous search through logs or attempts to simulate the context in debugging mode in order to reproduce the issue can become a bottleneck really quick in complex distributed systems. Same situation happens on the other side when you consume an API and have to deal with *'wtf is this'* error responses and we all know how painful it is.

Talking about distributed systems; another case of what I've encountered myself is having a microservice ecosystem with 5 individual services where each of them had its own error handling design. This is ok because in a microservice context each service can be written in different programming languages as long as they use a common ground for communication and maintain the API contract. The problem was that each of them returned different error response payloads. One of the microservices used 3 of them and we had to implement a REST client capable of handling 3 different error response schemas in a generic and uniform way in order to maintain the backwards compatibility of the service. Not fun!

A common pattern I see so far is teams don't spend enough time polishing the exception handling mechanism and consider it less important ending in scenarios where extra effort must be done which can cost the client money.

The Solution

Usually for scenarios where an API request needs to be fulfilled with business logic involving or not external calls you want to catch the specific database, http client or other unexpected exceptions, log the stack trace and throw your custom ones which are converted later on into prettier error responses.

As an example consider we want to fetch a TODO by id but we encounter *EntityNotFoundException* database exception. In my opinion is not a good idea to return this exception's message straight to the caller because it might contain sensitive information (e.g. the internal id of the entity) and instead use a custom one like 'TODO not found based on the given input', which has more meaning. Keep in mind the business team could have the requirement to return business-ish error messages in some cases so you will need to do the same thing anyway: wrapping the original exception into a custom one or directly throw your custom one with information inside from a try/catch block.

Another example is fetching a TODO by id which implies an external call to a metadata service let's say. If this call fails you don't want to return 'External call to metadata service failed' or the error message of the REST

client you are using because the caller doesn't need to know what's happening inside your system. You should return instead 'Something went wrong getting TODO' etc.

My point is, whenever you encounter an exception during an API request, you should always **map the exception to a custom one**, where possible of course, **which respects the error response schema being returned to the caller. Always!**

These being said, in an effort to standardize the API exception handling mechanism and create something generic that can be re-used and easily maintained I ended up with 4 different steps which I will explain in detail. The main idea was to create the mechanism in such a way that could be extracted at any point as a separate library without introducing breaking changes but to remain as flexible as possible. Let's begin!

Step 1. Error Response Schema

The first step implies defining a meaningful error response schema in order to decide what information we want to return in case of an API exception.

As an inspiration I used the **RFC 7807** devised by **Internet Engineering Task Force (IETF)** and created a custom one:

```
1  {
2      "code": "MethodArgumentNotValidException",
3      "message": "Bad Request",
4      "status": 400,
5      "timestamp": "2021-12-14T11:05:26.8972278",
6      "invalidParameters": [
7          {
8              "parameter": "age",
9              "message": "must be between 18 and 35"
10         }
11     ]
12 }
```

custom-error-schema.json hosted with ❤ by GitHub

[view raw](#)

Custom error schema based on RFC-7807

where:

1. **code** — a code that categorizes the exception
2. **message** — a human-readable explanation of the exception
3. **status** — the HTTP response code
4. **timestamp** — the timestamp when the exception occurred
5. **invalidParameters** — *the list of invalid parameters in case of multiple violations/constraints (optional)*

Step 2. Exception Policy

In this step we define a contract for our custom exceptions in order to have a standardized behavior across the API and provide some of the required properties for our error response schema. This contract can be a starting point for defining other policies also, as we will see later.

```
1  public interface ExceptionPolicy {  
2  
3      String getCode();  
4  
5      String getMessage();  
6  
7  }
```

ExceptionPolicy.java hosted with ❤ by GitHub

[view raw](#)

The structure of the base exception policy

I consider the **code** and the **message** being the mandatory information which must be present in each custom exception we want to define and as you already guessed there's an unspoken connection between the policy and the information we want to include in our error response schema.

Talking about custom exceptions, there are 2 types that I consider to be important when building an API:

- ***BusinessException*** — *thrown on each API request which involves business logic*
- ***ApplicationException*** — *thrown in all the other cases where we don't have to return something to the caller (background processing, listeners etc.)*

Because the base exception policy represents a starting point for other policies we can extend it in order to include additional information. In our case we need the HTTP status to be considered for *BusinessExceptions* also. So we define another custom policy:

```
1  public interface BusinessExceptionPolicy extends ExceptionPolicy {  
2  
3      HttpStatus getHttpStatus();  
4  
5  }
```

BusinessExceptionPolicy.java hosted with ❤ by GitHub

[view raw](#)

Custom business exception policy extending the base one

As you can see we just extend the base policy and create the additional method for returning an HTTP status also and yes, this represents a connection with our **status** property from the error response schema. Simple as that!

Step 3. Exception Classes and Reasons

A common pattern I saw is to define a custom exception for each case when we can't fulfil a business logic. For example:

- *ToDoNotFound*
- *ToDoNotCreated*
- *ToDoNotDeleted*
- *ToDoNotUpdated*

This works ok but **the main problem is the increasing number of classes, duplicated code and maintainability**. I remember I saw at some point 10 different exception classes used inside a method which were thrown based on different checks done for that particular business logic. Let's be honest, even if you create a base class for them to avoid duplication you still end up with a lot of classes doing the same thing: carrying an exception message and maybe something else.

The point is you can loose track of them really fast and if you want to include additional information inside some of them then you need to do

monkey work and duplicate because you don't want to break the other classes by putting the new information inside the base class.

In order to solve the mentioned issues we can stick to only one class named *BusinessException* which looks like this:

```
1  @Getter
2  @Setter
3  public class BusinessException extends RuntimeException implements BusinessExceptionPolicy {
4
5      protected final String code;
6      protected final String message;
7      protected final HttpStatus httpStatus;
8
9
10     public BusinessException(final BusinessExceptionReason reason) {
11         this.code = reason.getCode();
12         this.message = reason.getMessage();
13         this.httpStatus = reason.getHttpStatus();
14     }
15
16     public BusinessException(final BusinessExceptionReason reason, final HttpStatus overridingHttp
17         this.code = reason.getCode();
18         this.message = reason.getMessage();
19         this.httpStatus = overridingHttpStatus;
20     }
21
22     public BusinessException(final BusinessExceptionReason reason, final Object... parameters) {
23         if (parameters != null) {
```

```

24         this.message = format(reason.getMessage(), parameters);
25     } else {
26         this.message = reason.getMessage();
27     }
28
29     this.code = reason.getCode();
30     this.httpStatus = reason.getHttpStatus();
31 }
32
33 @Override
34 public String getLocalizedMessage() {
35     return getMessage();
36 }
37
38 public String toString() {
39     return format("BusinessException(code=%s, message=%s, httpStatus=%s)", this.getCode(), t
40         this.getHttpStatus().value());
41 }
42 }

```

BusinessException.java hosted with ❤ by GitHub

[view raw](#)

Custom exception class for business related scenarios

What is that *BusinessExceptionReason* present in constructors you may ask. It's an enum class which holds information about the reason why the *BusinessException* was thrown and looks like this:

```

1 @Getter

```

```
2  @AllArgsConstructor
3  public enum BusinessExceptionReason implements BusinessExceptionPolicy {
4
5      TODO_NOT_FOUND_BY_EXT_REF("TODO not found based on the given external reference", HttpStatus
6
7      private final String code = BusinessExceptionReason.class.getSimpleName();
8      private final String message;
9      private final HttpStatus httpStatus;
10
11 }
```

BusinessExceptionReason.java hosted with ❤ by GitHub

[view raw](#)

Custom business exception reasons

As a developer I wanted a simpler way of controlling the messages and HTTP statuses returned to the caller whenever I throw a business exception. This way I can add, replace, delete and update everything related to business exceptions in one single place, here.

Do I want to return `HttpStatus.BAD_REQUEST` in case a TODO is not found by the external reference? I change from `NOT_FOUND` to `BAD_REQUEST` and done; 3 seconds change!

Do I want to change the message and include the external reference value also? I add `%s` in the message like this `TODO not found based on the given`

external reference %s and then I use the *BusinessException*'s constructor accepting the varargs argument `Object...parameters`. That's it!

Here are some examples:

```
1 // basic use without optional input paremeters
2 throw new BusinessException(TODO_NOT_FOUND_BY_EXT_REF)
3
4 // basic use with optional input parameters
5 throw new BusinessException(TODO_NOT_FOUND_BY_EXT_REF, "ext-ref-12345678");
6
7 // basic use where the original reason's HTTP status is overrided with the given one
8 throw new BusinessException(TODO_NOT_FOUND_BY_EXT_REF, HttpStatus.FORBIDDEN);
```

BusinessExceptionExamples.java hosted with ❤ by GitHub

[view raw](#)

Examples using different constructors of *BusinessException*

This gives us flexibility, maintainability and more important an easier way of throwing and managing business related exceptions. Keep in mind that **both of them** (*BusinessException* and *BusinessExceptionReason*) **must adhere to the same exception policy!**

Step 4. Global Exception Handler

The last step is to define the global exception handler. This is the `@ControllerAdvice` annotated class as we know it and looks like this:

```
1  @Order(Ordered.HIGHEST_PRECEDENCE)
2  @ControllerAdvice
3  @Slf4j
4  public class GlobalApiExceptionHandler extends ResponseEntityExceptionHandler {
5
6      @ExceptionHandler(value = {Exception.class})
7      public ResponseEntity<Object> handleUncaughtException(final Exception ex, final ServletWebRe
8          log(ex, request);
9      final ErrorResponseDto errorResponseDto = build(Exception.class.getSimpleName(),
10          HttpStatus.INTERNAL_SERVER_ERROR.getReasonPhrase(),
11          HttpStatus.INTERNAL_SERVER_ERROR);
12      return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(errorResponseDto);
13  }
14
15      @ExceptionHandler({BusinessException.class})
16      public ResponseEntity<Object> handleCustomUncaughtBusinessException(final BusinessException
17          log(ex, request);
18      final ErrorResponseDto errorResponseDto = build(ex.getCode(), ex.getMessage(), ex.getHttp
19      return ResponseEntity.status(ex.getHttpStatus()).body(errorResponseDto);
20  }
21
22      // the other methods omitted
23
24  }
```

GlobalApiExceptionHandler.java hosted with ❤ by GitHub

[view raw](#)

This handler is responsible of catching uncaught exceptions thrown inside our API. It provides a lot of methods that can be overridden for providing custom behavior and can be extended to deal with custom exceptions also. More [details here](#).

Note: The other methods from the handler are omitted for simplicity but as I already mentioned you can find the [code here](#).

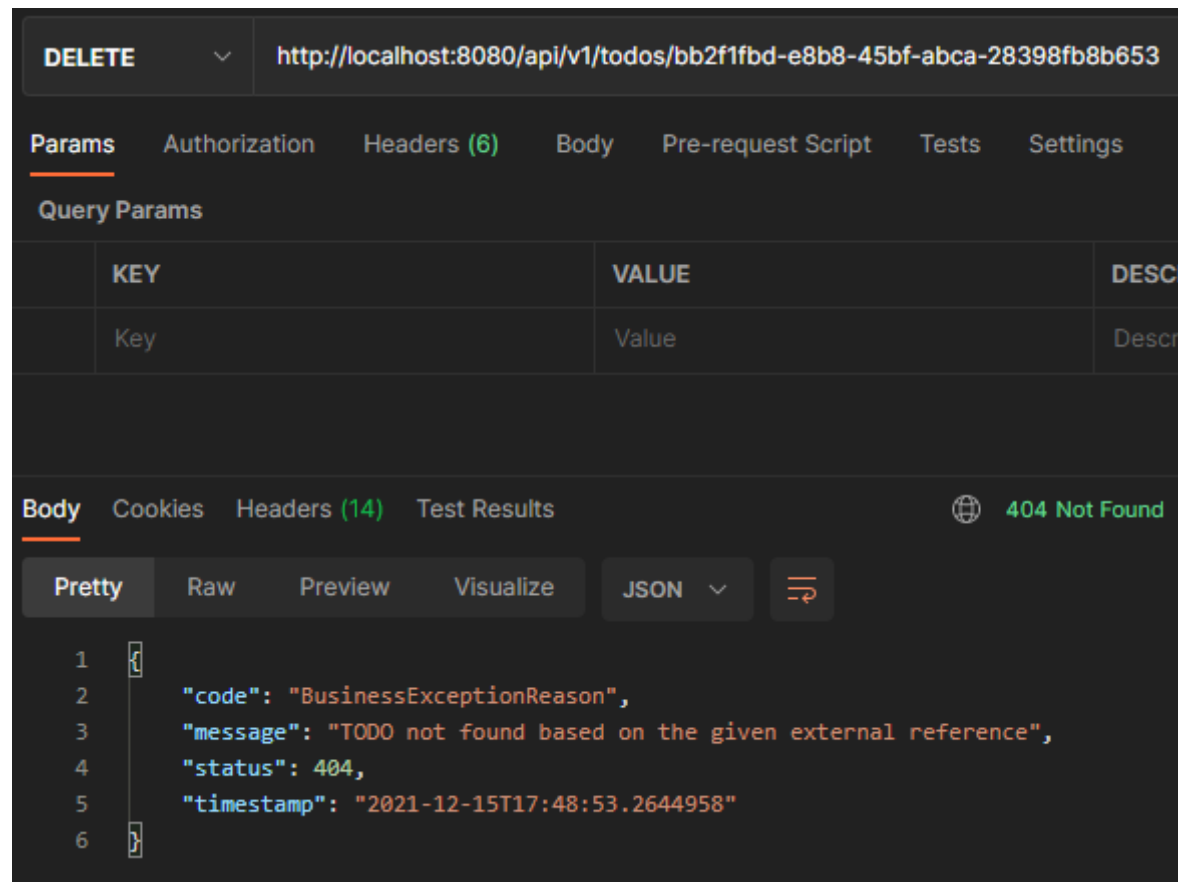
What we have to do is to simply declare a method annotated with `@ExceptionHandler` which should handle our custom *BusinessException* when thrown and extract the required properties in order to create the error response schema. Notice the returned HTTP status code is the one coming from the exception and this means whenever we change the HTTP status of an exception reason, the change will be reflected automatically without any additional intervention to our code base!

The `handleUncaughtException` acts as a fallback mechanism in case we don't have a method inside the handler to deal with an uncaught exception. For example a *NullPointerException* can occur but it will be caught by the method which will return an Internal Server Error response. This way we are bullet proof and we make sure we always return the same error response schema no matter what uncaught exception was thrown.

Demo

Here are some screenshots with different error responses for a dummy TODO API:

1. Delete non-existing TODO by external reference

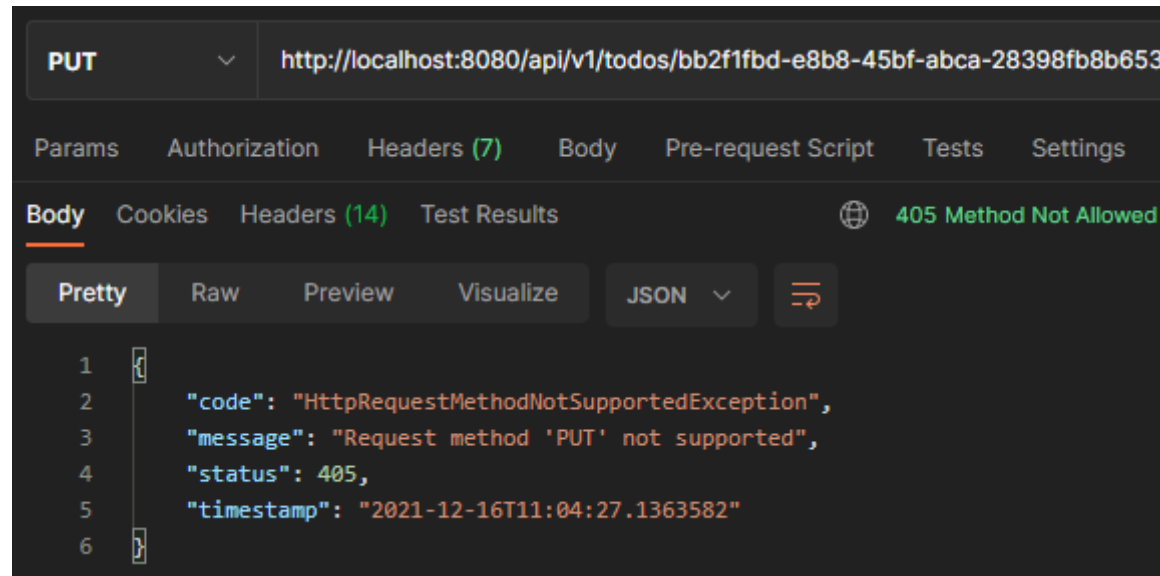


2. Create TODO with invalid 'title'

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/v1/todos`. The request body is a JSON object with an empty string for the 'title' field: `{ "title": "" }`. The response is a 400 Bad Request, with the body showing a standardized error message:

```
{
  "code": "MethodArgumentNotValidException",
  "message": "Bad Request",
  "status": 400,
  "timestamp": "2021-12-15T17:47:53.1676255",
  "invalidParameters": [
    {
      "parameter": "title",
      "message": "size must be between 1 and 255"
    },
    {
      "parameter": "title",
      "message": "must not be empty"
    }
  ]
}
```

3. HTTP request method not allowed



Conclusion

As always please keep in mind this approach might or might not suit your project context or needs and I'm not in the position to say there's no other way to do it differently or better. I really hope you enjoyed it and had fun reading it.

Stay safe and remember you can find the [code here!](#)

Spring Boot Programming API Exception Handling

About Write Help Legal